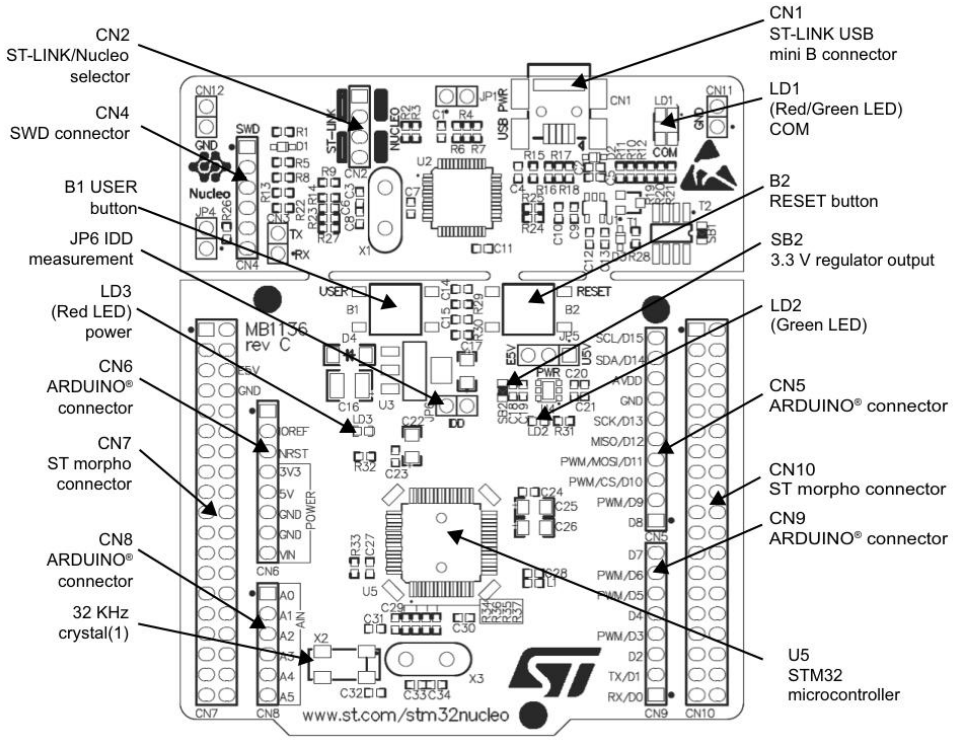
EE426 / 626 Microprocessor Applications

Reset and Clock Control (RCC)



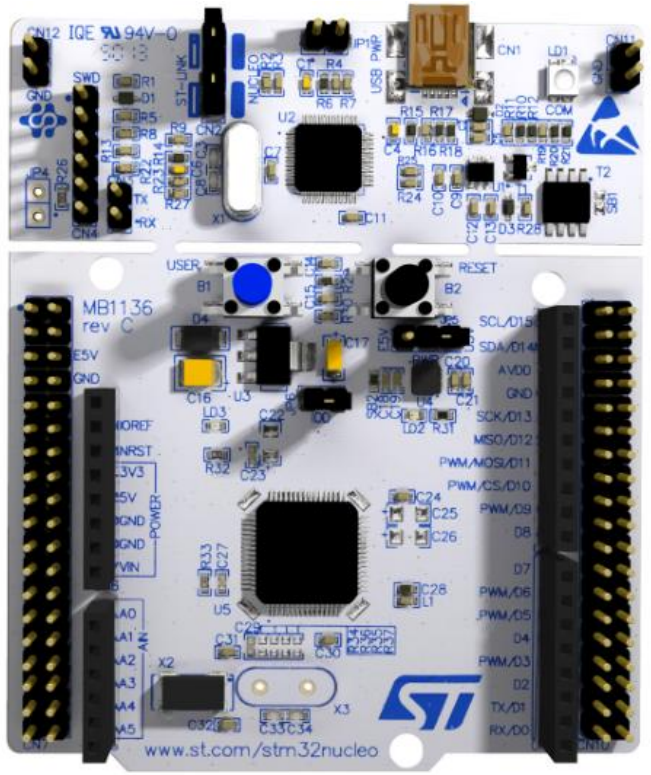
EE426 / 626 Microprocessor Applications

Nucleo-64 STM32F446RE



Fair use, STMicroelectronics NV
UM1724 Rev 17, (STM32 Nucleo-64 boards, MB1136)

Nucleo-64 STM32F446RE



Fair use, STMicroelectronics NV
UM1724 Rev 17, (STM32 Nucleo-64 boards, MB1136)

EE426 / 626 Microprocessor Applications

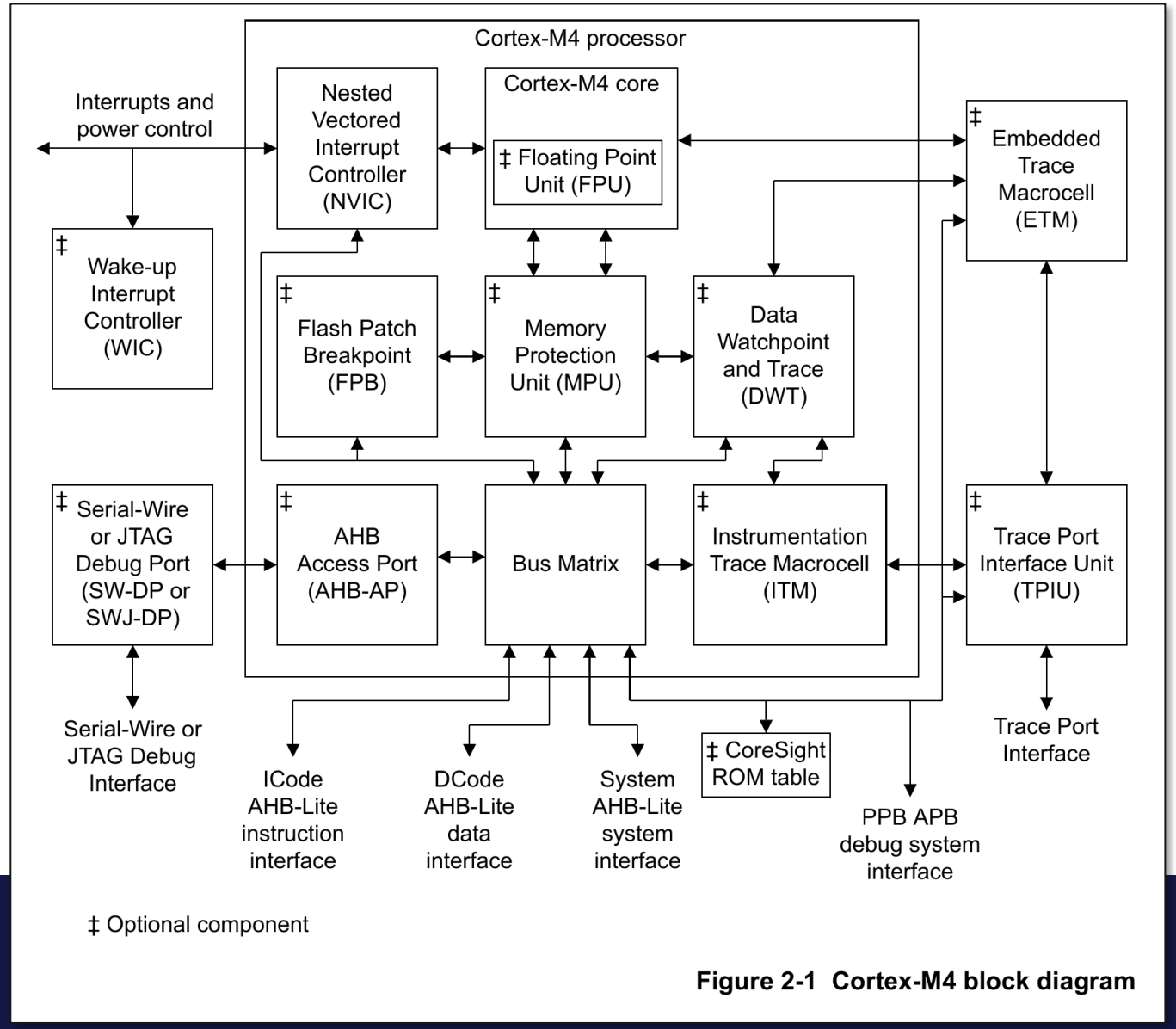
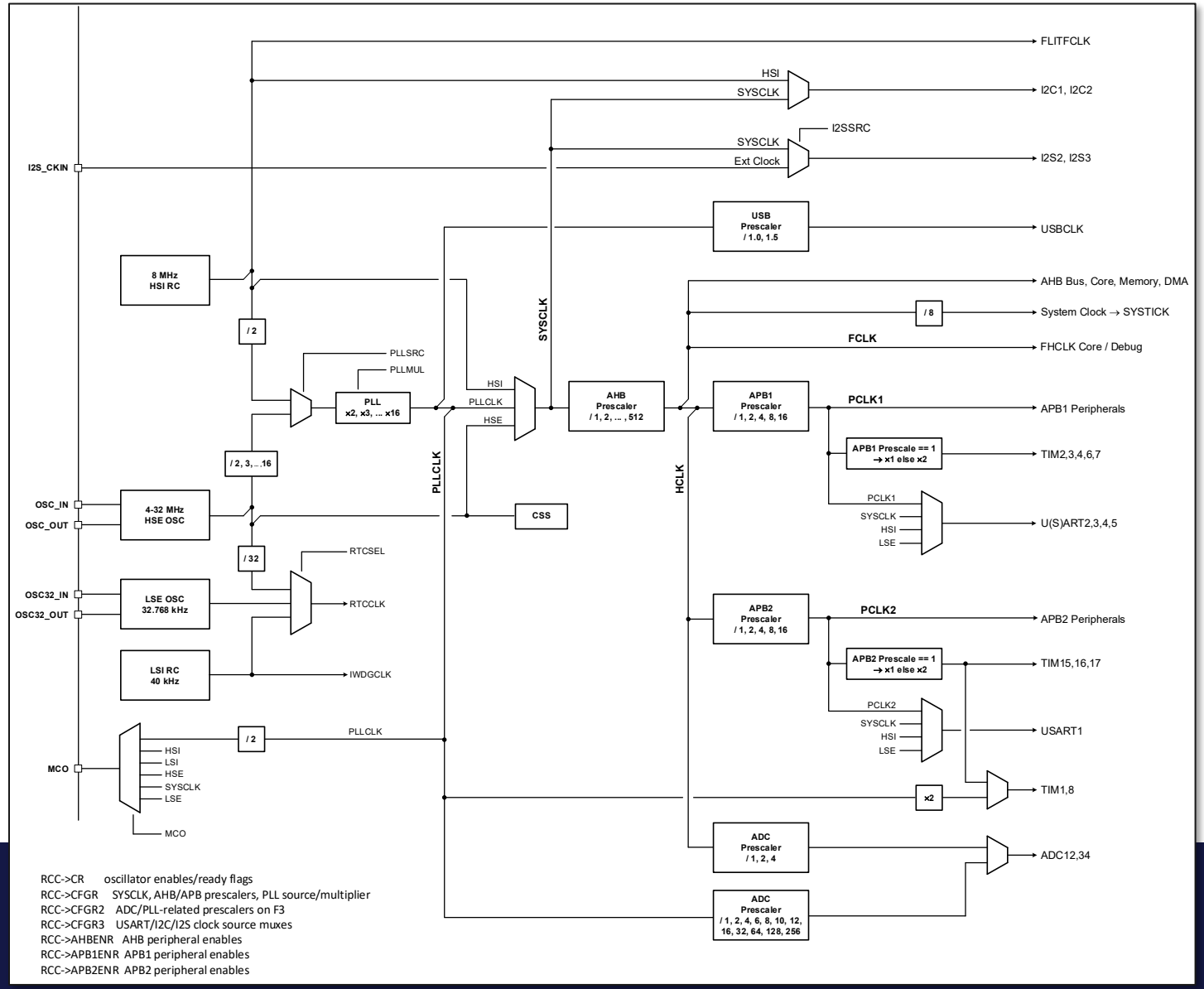


Figure 2-1 Cortex-M4 block diagram

ST Microelectronics
STM32F446RE

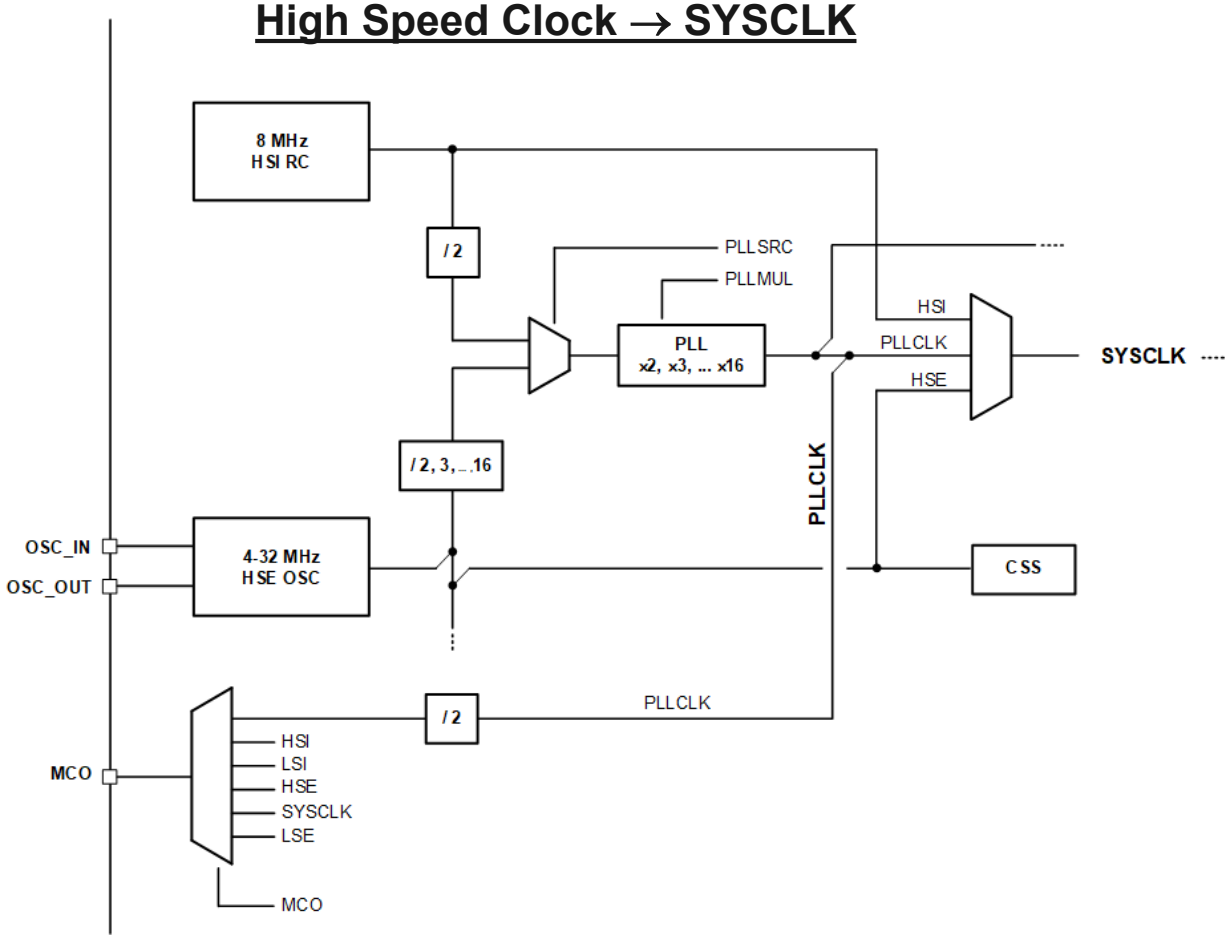
EE426 / 626 Microprocessor Applications



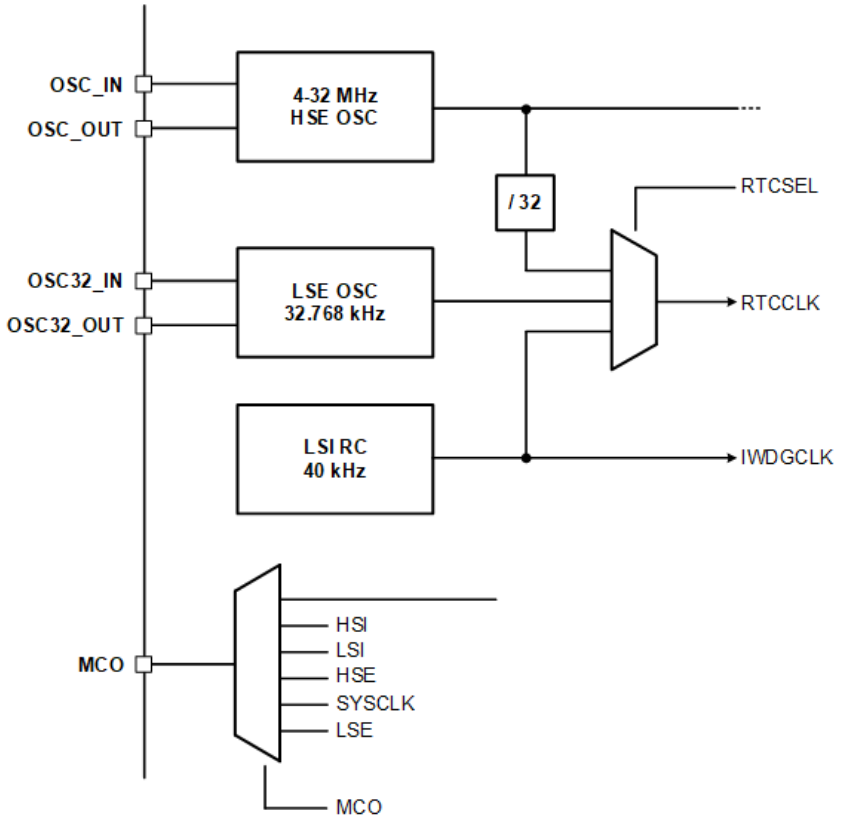
STM32F446RE
Reset and Clock Control (RCC)

EE426 / 626 Microprocessor Applications

High Speed Clock → SYSCLK



Low Speed Clock → RTCCLK and Watchdog



EE426 / 626 Microprocessor Applications

0x00	RCC_CR	Clock control register
0x04	RCC_PLLCFGR	PLL configuration register
0x08	RCC_CFGR	Clock configuration register
0x0C	RCC_CIR	Clock interrupt register
0x10	RCC_AHB1RSTR	AHB1 peripheral reset register
0x14	RCC_AHB2RSTR	AHB2 peripheral reset register
0x18	RCC_AHB3RSTR	AHB3 peripheral reset register
0x20	RCC_APB1RSTR	APB1 peripheral reset register
0x24	RCC_APB2RSTR	APB2 peripheral reset register
0x30	RCC_AHB1ENR	AHB1 peripheral clock enable register
0x34	RCC_AHB2ENR	AHB2 peripheral clock enable register
0x38	RCC_AHB3ENR	AHB3 peripheral clock enable register
0x40	RCC_APB1ENR	APB1 peripheral clock enable register
0x44	RCC_APB2ENR	APB2 peripheral clock enable register



EE426 / 626 Microprocessor Applications

0x50	RCC_AHB1LPENR	AHB1 peripheral clock enable
0x54	RCC_AHB2LPENR	AHB2 peripheral clock enable
0x58	RCC_AHB3LPENR	AHB3 peripheral clock enable
0x60	RCC_APB1LPENR	APB1 peripheral clock enable
0x64	RCC_APB2LPENR	APB2 peripheral clock enable
0x70	RCC_BDCR	Backup domain control register
0x74	RCC_CSR	Clock control and status register
0x80	RCC_SSCGR	Spread spectrum clock generation register
0x84	RCC_PLLI2SCFGR	PLLI2S configuration register
0x88	RCC_PLLSAICFGR	PLL configuration register (PLLSAI)
0x8C	RCC_DCKCFGR	Dedicated clock configuration register
0x90	CKGATENR	Clocks gated enable register
0x94	DCKCFGR2	Dedicated clocks configuration register 2

See RM0390 – STM32F446xx Advanced ARM Based 32-Bit MCUs

Chapter 6 Reset and clock control (RCC)



EE426 / 626 Microprocessor Applications

Configuring HSE — CMSIS

Step 1. Turn on the HSE oscillator by setting its enable bit in RCC_CR.
Assumes RCC_PLLCFGR has already been set up elsewhere — PLL source, and the M/N/P dividers since that has to be configured before the PLL is turned on in Step 3.)

```
RCC->CR |= RCC_CR_HSEON;                    // (1 << 16)
```

Step 2. Poll the HSE-ready flag and wait here until the hardware confirms the external crystal has actually stabilized — HSE doesn't start oscillating cleanly the instant it's enabled.

```
while (!(RCC->CR & RCC_CR_HSERDY));         // (1 << 17)
```

Step 3. Turn on the main PLL now that its configuration (source = HSE, dividers) is already in place from the earlier RCC_PLLCFGR setup.

```
RCC->CR |= RCC_CR_PLLON;                    // (1 << 24)
```

Step 4. Poll the PLL-lock flag and wait until the PLL has settled onto its target frequency.
Like HSE, it isn't instantly stable the moment it's enabled.

```
while (!(RCC->CR & RCC_CR_PLLRDY));         // (1 << 25)
```

Step 5. Switch the system clock source over to the PLL output by writing the SW select bits in RCC_CFGR.

```
RCC->CFGR |= RCC_CFGR_SW_PLL;               // (0x2 << 0)
```



EE426 / 626 Microprocessor Applications

Configuring HSE — CMSIS

Step 6. Poll the SWS status field and wait until the hardware confirms the switch actually completed.
The clock mux doesn't change over instantaneously, so software must not assume it's done yet.

```
while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);    // (0x3 << 2) != (0x2 << 2)
```

Step 7. Set the APB1 prescaler to divide system clock by 4, since APB1 has a hard ceiling of 45 MHz that the full SYSCLK would exceed.

```
RCC->CFGR |= RCC_CFGR_PPRE1_DIV4;                          // (0x5 << 10)
```

Step 8. Set the APB2 prescaler to divide by 2, since APB2's ceiling is higher (90 MHz) but still needs a division off the SYSCLK.

```
RCC->CFGR |= RCC_CFGR_PPRE2_DIV2;                          // (0x4 << 13)
```

Step 9. Enable the clock to GPIOA in RCC_AHB1ENR
This is the same line from today's worked example, now shown in its natural home alongside the rest of the clock tree setup.

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;                       // (1 << 0)
```

Note ‘|=‘ is safe here because reset defaults are 0. If not initializing from reset then registers need to be modified using read/write/modify.



EE426 / 626 Microprocessor Applications

Configuring LSE — CMSIS

Step 1. Enable the clock to the PWR peripheral, since the next step needs to write to a PWR register (PWR_CR), and like GPIOA earlier, PWR is gated off until its bit is set in the APB1 clock enable register.

```
RCC->APB1ENR |= RCC_APB1ENR_PWREN;                    // (1 << 28)
```

Step 2. Disable backup domain write protection. The backup domain (RTC + its clock control bits in RCC_BDCR) is protected from accidental writes by default — so it must be explicitly unlocked before RCC_BDCR can be modified.

```
PWR->CR |= PWR_CR_DBP;                                // (1 << 8)
```

Step 3. Turn on the LSE oscillator by setting its enable bit in RCC_BDCR.

```
RCC->BDCR |= RCC_BDCR_LSEON;                          // (1 << 0)
```



EE426 / 626 Microprocessor Applications

Configuring LSE — CMSIS

Step 4. Poll the LSE-ready flag and wait here until the 32.768 kHz crystal has actually stabilized.
Same pattern as HSE, but crystals this slow can take noticeably longer (often hundreds of milliseconds) to start up cleanly.

```
while (!(RCC->BDCR & RCC_BDCR_LSERDY));                    // (1 << 1)
```

Step 5. Select LSE as the clock source for the RTC by writing 01 into the RTCSEL[1:0] field.
This only chooses which source the RTC will use, it doesn't yet turn the RTC on.

```
RCC->BDCR |= RCC_BDCR_RTCSEL_0;                             // (1 << 8)
```

Step 6. Enable the RTC itself by setting RTCEN
Until this bit is set, the RTC counter isn't actually clocked even though its source is now selected.

```
RCC->BDCR |= RCC_BDCR_RTCEN;                                // (1 << 15)
```

Note '|=' is safe here because reset defaults are 0. If not initializing from reset then registers need to be modified using read/write/modify.



EE426 / 626 Microprocessor Applications

STM32CubeMX

